

Name:

Roll no.:

Div

Class:

Batch:

Practical no. 1:

Step no.1: Open folder in any name.

Step no.2: Open Command Prompt (cd).

Step no.3: Type command:

1: dir

2: cd downloads

3: mkdir (Folder name)

4: npx create-react-app (Folder name)

And running this process :

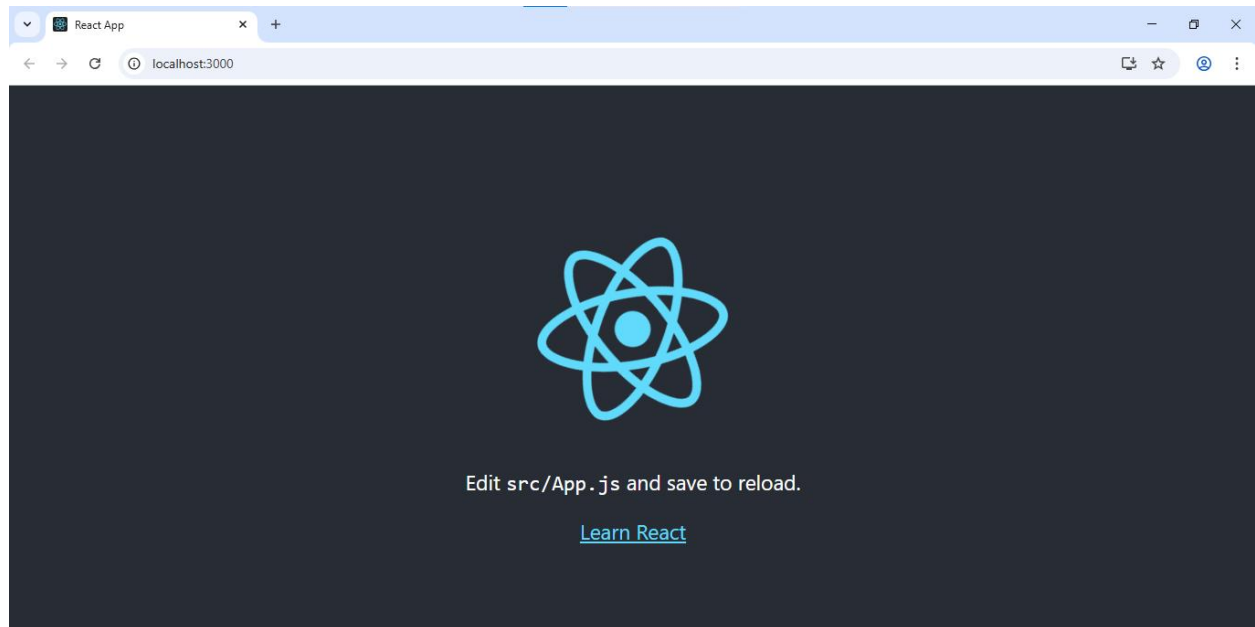
- **Go to V.S code and open your folder.**
- **Open App.js file**

App.js code:

```
import logo from './logo.svg';
import './App.css';
function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>
        <a
          className="App-link"
          href="https://reactjs.org"
          target="_blank"
          rel="noopener noreferrer"
        >
          Learn React
        </a>
      </header>
    </div>
  );
}
export default App;
```

- **Run terminal**
- **Type command: npm start**
- **Open any Browser**

Output:



Name:
Batch

Class:

Div

Practical no. 2: JSX and component creation using props.

1. Create a file greeting.js write this code, in greeting.js

```
function Greeting(props){  
  return <p>Hello, {props.name}!  
  Nice to see you. </p>;  
}  
export default Greeting;
```

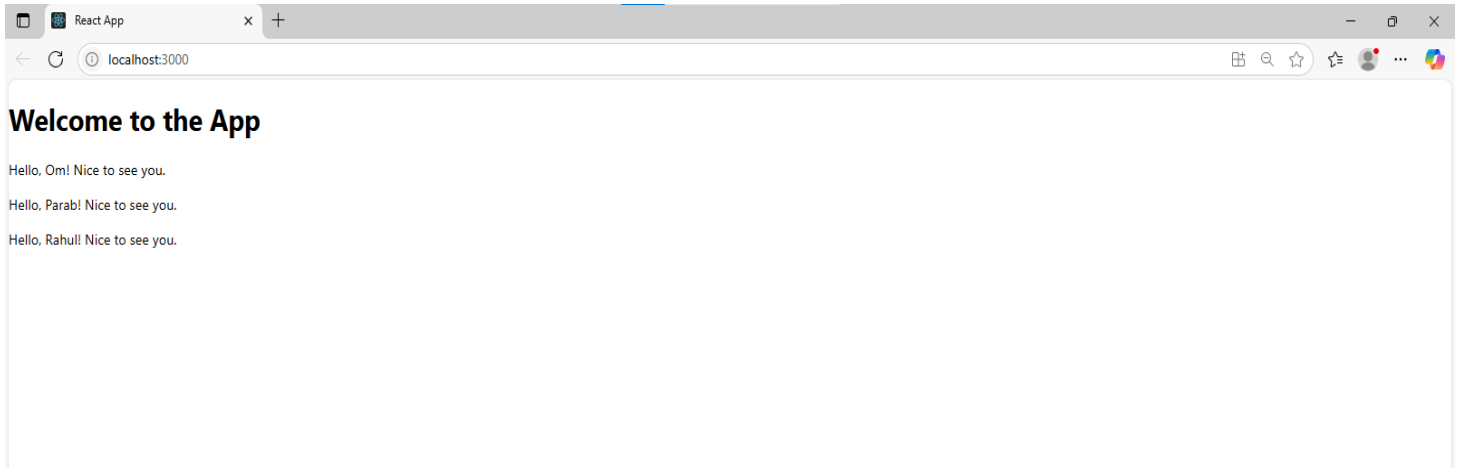
2. App.js

Write this code in App.js and App.js

```
import React from 'react';  
import Greeting from './Greeting';  
function App() {  
  return (  
    <div>  
      <h1>Welcome to the App</h1>  
      <Greeting name="Om"/>  
      <Greeting name="Parab"/>  
      <Greeting name="Rahul"/>  
    </div>  
  );  
}  
export default App;
```

3. Run this code `npm start`

Output:



Name:
Batch

Class:

Div

Practical no. 3: State management in React using use state: Building counter app

```
import React, { useState } from 'react';

function CounterApp() {

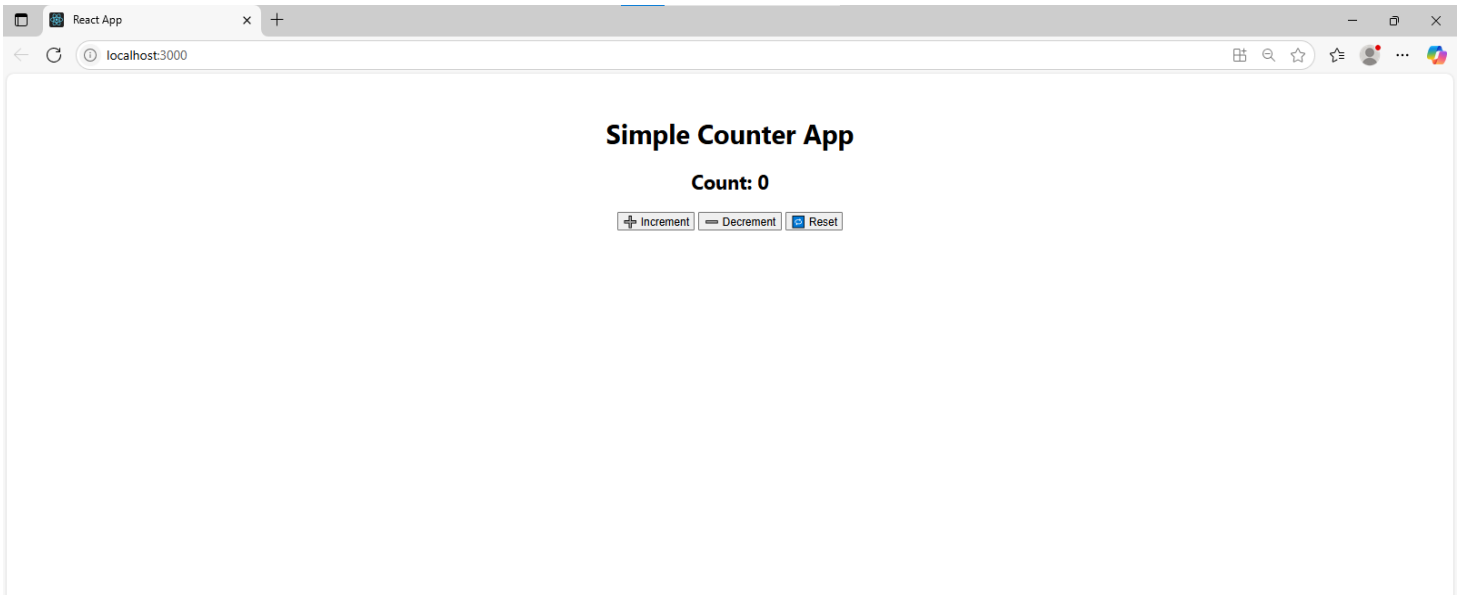
  const [count, setCount] = useState(0);

  const increment = () => setCount(count + 1);
  const decrement = () => setCount(count - 1);
  const reset = () => setCount(0);

  return (
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
      <h1>Simple Counter App</h1>
      <h2>Count: {count}</h2>
      <button onClick={increment}>➕ Increment</button>{' '}
      <button onClick={decrement}>➖ Decrement</button>{' '}
      <button onClick={reset}>↺ Reset</button>
    </div>
  );
}

export default CounterApp;
```

Output:



Name:

Roll no.:

Div:

Batch:

Class: SYBCA

Practical no.4: Event Handling in React: Managing Button Clicks and Input Changes.

```
import React, { useState } from 'react';

function EventHandlingExample() {
  const [name, setName] = useState("");
  const [message, setMessage] = useState("");

  const handleInputChange = (e) => {
    setName(e.target.value);
  };

  const handleButtonClick = () => {
    setMessage(`Hello, ${name}!`);
  };

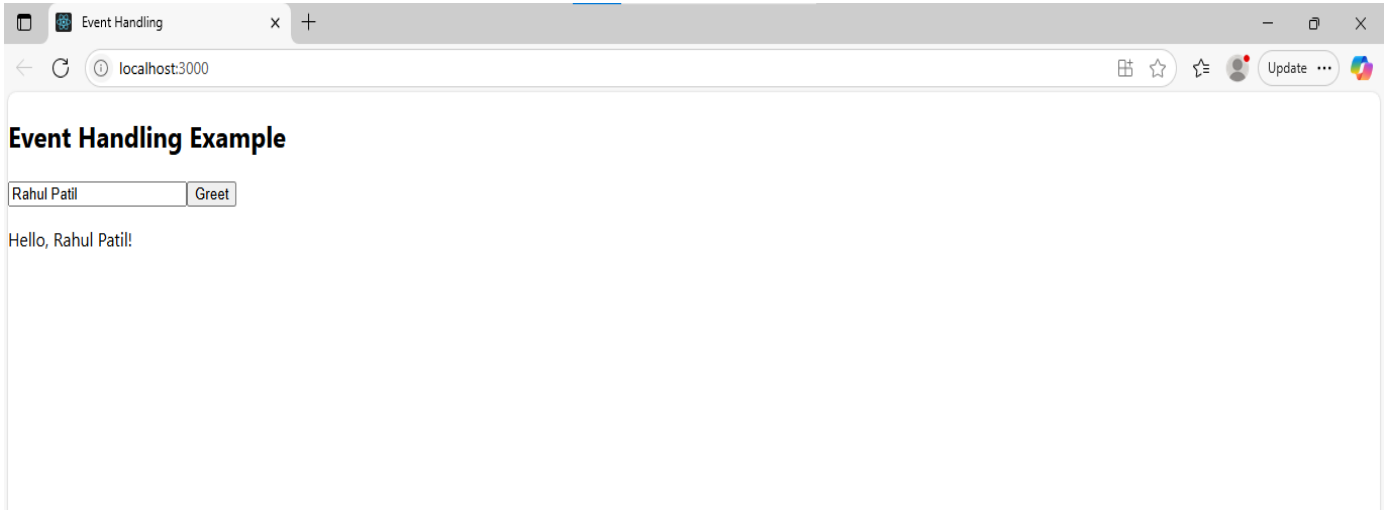
  return (
    <div>
      <h2>Event Handling Example</h2>

      <input
        type="text"
        placeholder="Enter your name"
        value={name}
        onChange={handleInputChange}/>

      <button onClick={handleButtonClick}>Greet</button>
      <p>{message}</p>
    </div>
  );
}

export default EventHandlingExample;
```

Output:



Name:

Roll no.:

Div:

Batch:

Class: SYBCA

Practical no.5: Rendering Dynamic Lists in React Using Array Mapping and Key Props

```
import React from 'react';

const FruitsList = () => {

  const fruits = ['Pineapple', 'Jackfruit', 'Watermelon'];

  return (

    <ul>

      <h1>FRUITS LIST</h1>

      {fruits.map((fruit, index) => (

        <li key={index}>{fruit}</li>

      )))}

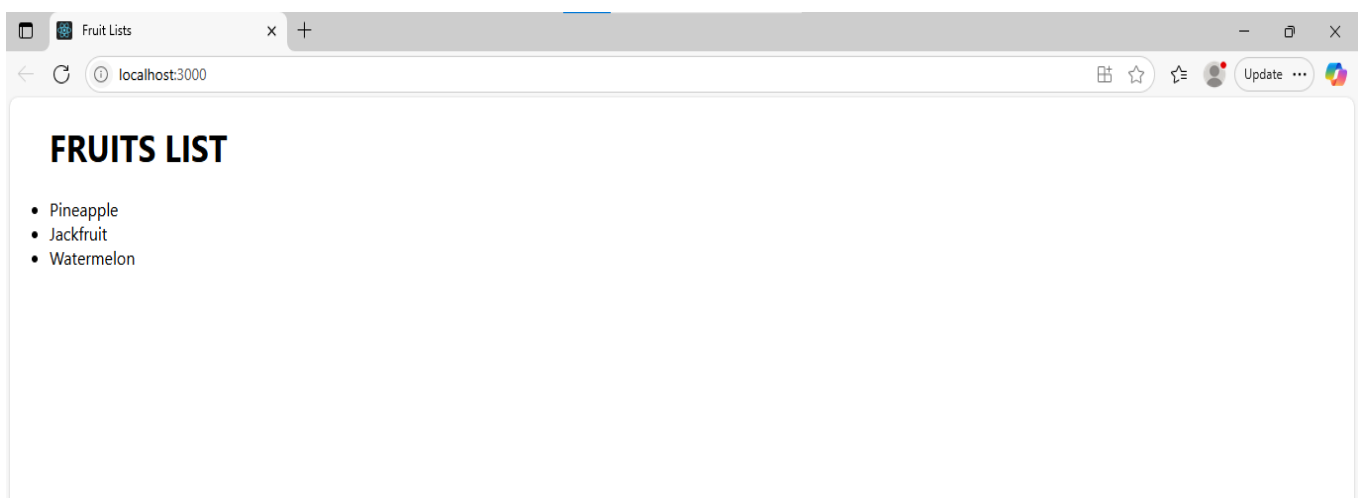
    </ul>

  );

};

export default FruitsList;
```

Output:



Name:

Roll no.:

Div:

Batch:

Class: SYBCA

Practical no.6: Building Controlled Forms in React: Managing State and Implementing Basic Validation

```
import { useState } from 'react';
export default function MyForm() {

  const [inputValue, setInputValue] = useState("");
  const [inputError, setInputError] = useState(null);

  function handleInputChange(event) {
    const value = event.target.value;
    setInputValue(value);

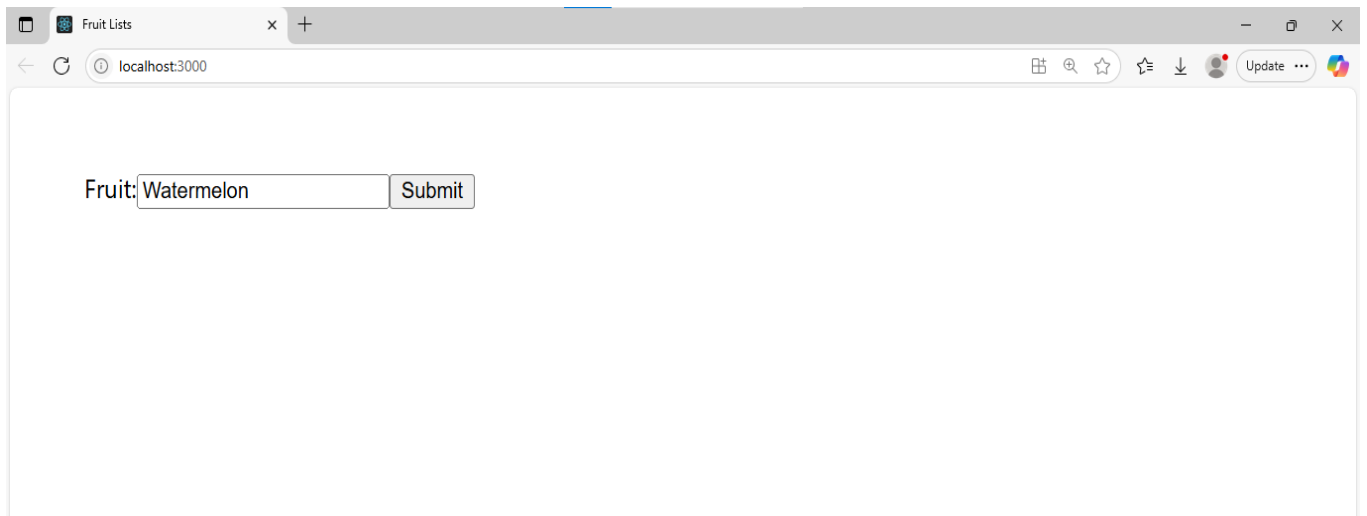
    if (value.length < 5) {
      setInputError('Input must be at least 5 characters');
    } else {
      setInputError(null);
    }
  }

  function handleSubmit(event) {
    event.preventDefault();

    if (inputValue.length >= 5) {
    } else {
      setInputError('Input must be at least 5 characters');
    }
  }

  return (
    <form onSubmit={handleSubmit} style={{margin:"50px"}}>
      <label>
        Fruit:
        <input type="text" value={inputValue} onChange={handleInputChange} />
      </label>
      {inputError && <div style={{ color: 'red' }}>{inputError}</div>}
      <button type="submit">Submit</button>
    </form>
  );
}
```

Output:



The screenshot shows a web browser window with a single tab titled "Fruit Lists". The address bar displays "localhost:3000". The browser's toolbar includes navigation buttons (back, forward, refresh), a search icon, a star icon, a download icon, and a user profile icon with an "Update ..." button. The main content area of the browser displays a form with the label "Fruit:" followed by a text input field containing the word "Watermelon" and a "Submit" button.

Name:

Roll no.:

Div:

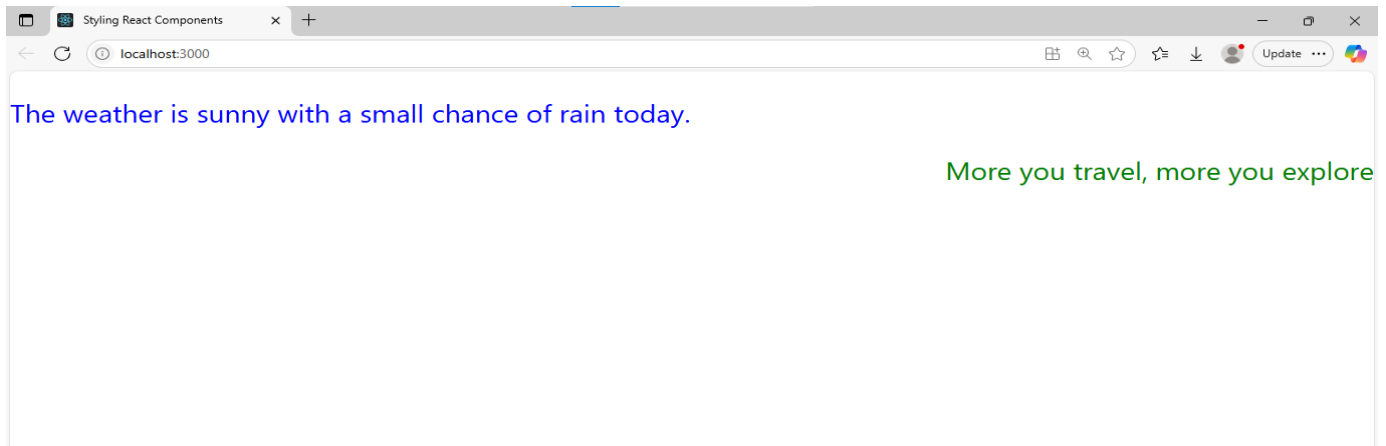
Batch:

Class: SYBCA

Practical no.7: Styling React Components: Implementing CSS Modules and Inline Styles

```
import React from "react";
const pStyle = {
  fontSize: "16px",
  color: "blue"
};
export default function App() {
  return (
    <>
      <p style={pStyle}>
        The weather is sunny with a small chance of rain today.
      </p>
      <p style={{ ...pStyle, color: "green", textAlign: "right" }}>
        More you travel, more you explore </p>
    </>
  );
}
```

Output:



Name:

Roll no.:

Div:

Batch:

Class: SYBCA

Practical no.8: Conditional styling in React using both CSS Modules and inline styles. (Toggle Button with Conditional Styling).

toggle.css

```
.container {  
  padding: 30px;  
  text-align: center;  
  border: 2px solid #ccc;  
  width: 300px;  
  margin: 50px auto;  
  border-radius: 10px;  
}
```

```
.label {  
  font-size: 24px;  
  margin-bottom: 20px;  
  display: block;  
}
```

App.js

```
import React, { useState } from 'react';  
import styles from './toggle.css';
```

```
const ToggleButton = () => {  
  const [isOn, setIsOn] = useState(false);
```

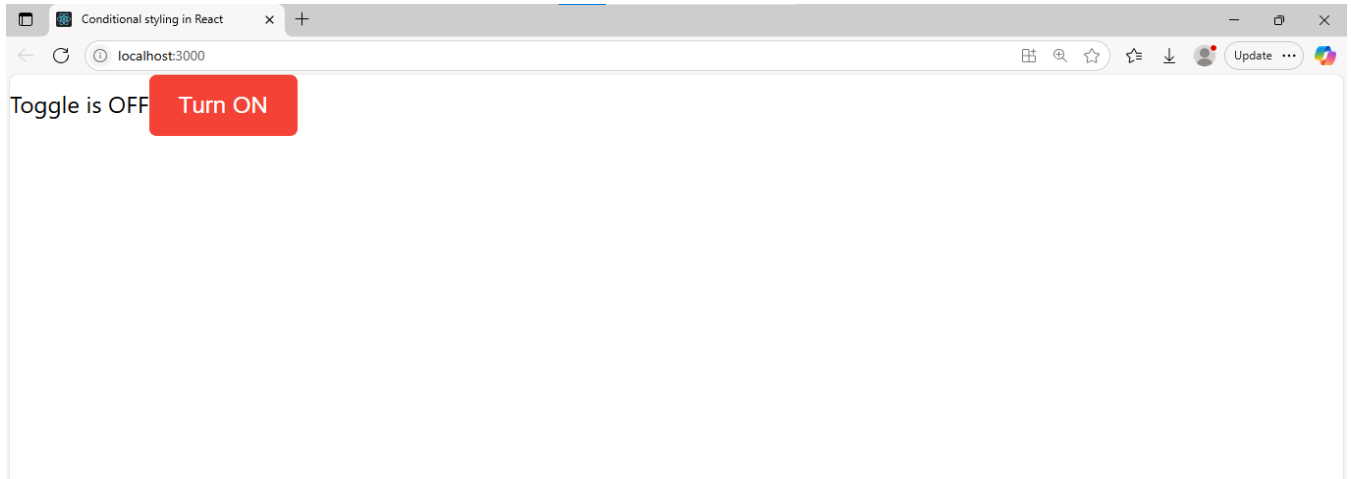
```
  const buttonStyle = {  
    backgroundColor: isOn ? '#4CAF50' : '#f44336',  
    color: 'white',  
    padding: '12px 20px',  
    border: 'none',  
    borderRadius: '5px',  
    fontSize: '16px',  
    cursor: 'pointer',  
    transition: 'background-color 0.3s ease',  
  };
```

```
  return (  
    <div className={styles.container}>  
      <span className={styles.label}>Toggle is {isOn ? 'ON' : 'OFF'}</span>  
      <button style={buttonStyle} onClick={() => setIsOn(!isOn)}>  
        {isOn ? 'Turn OFF' : 'Turn ON'}  
      </button>  
    </div>  
  );
```

```
        </button>
      </div>
    );
  };

export default ToggleButton;
```

Output:



1. setup the Backend (Node.js + Express.js)

Open cmd

Write this commands:-

- mkdir fullstack-app
- cd fullstack-app
- mkdir backend
- cd backend
- npm init -y
- npm install express cors
- code .

2. Install Express

npm install express cors

3. Create a server.js

Type this code

```
const express = require('express');
```

```
const cors = require('cors');
```

```
const app = express();
```

```
const PORT =5000;
```

```
app.use(cors());
```

```
app.use (express.json());
```

```
//Dummy api
```

```
app.get('/api/message',(req,res)=>{
```

```
    res.json({message:"Hello from the backend!"});
```

```
});
```

```
app.listen(PORT,()=>{
```

```
console.log(`server running on http://localhost:${PORT}`);  
});
```

4. Run the backend server

CMD command: **node server.js**

Visit: **http://localhost:5000/api/message**

Output:

